Chapter 6 · Files, indexes and B+ trees

Imagine paper records stored in boxes. Opening a box has a cost. An index is a smaller directory telling us which box to open. A tree node can hold many directory entries, so we can reach a record after opening relatively few boxes.

Vocabulary. A block/page is a unit of storage transfer in the stated model. Blocking factor is records per block. A search key is the indexed attribute set; it need not be a candidate key. A dense index has entries covering every indexed search-key occurrence or distinct value as specified by its layout. A sparse index omits some search-key entries and relies on ordering. A clustering index concerns the data's physical grouping. Use the question's terminology and layout where textbooks differ.

R27 Blocking factor and storage size

For fixed-length unspanned records of size r in blocks of size B, after any stated overhead, bfr = floor(B/r). For N records, blocks = ceil(N/bfr). Records cannot straddle blocks in this model. Spanned or variable-length records require their own packing rule. Use actual sizes and distinguish bits from bytes and KB conventions.

Example: 100 records of 100 bytes in 1024-byte blocks need floor(1024/100)=10 records per block and ten blocks. Do not use ceil(10000/1024), which assumes different packing.

Worked example W27. For 101 unspanned records of 100 bytes in 1024-byte blocks, floor(1024/100)=10 records fit per block. Ten blocks hold 100 records; the last record requires an eleventh. Dividing total bytes by block size and rounding gives 10, but incorrectly packs records across boundaries. With a 50-byte header in every block, usable bytes become 974, so only 9 records fit and ceil(101/9)=12 blocks are needed. A heap file has no search-key order; an ordered file can support ordered searching; a hash-organized file places records by a hash value. Those organization choices affect access, not the arithmetic packing rule.

Check before moving on. Use floor for capacity and ceil for the number of containers. Mark usable bytes after overhead.

R28 Single and multilevel index size

Identify whether the lowest-level entries represent records, distinct values or data blocks. Write the bytes in one entry, including keys and pointers. Compute entries per index block with floor. Use ceil to calculate the blocks required. For multilevel indexing, repeat with one entry per block below until the top fits in one block. Track the entry layout at each level and count the requested levels, not data blocks by accident.

Worked example W28. 10,000 records of 100 bytes in 1000-byte blocks need 1000 data blocks. A sparse lowest-level index has one 20-byte anchor entry per data block, so 50 entries fit per index block. Its 1000 entries need 20 blocks. A second level has 20 anchors and fits in one block. There are 2 index levels and, with no cache, a lookup reads 2 index blocks plus 1 data block. If the lowest level is instead dense with one entry per record, it needs 200 blocks, then 4 blocks, then 1 root block: 3 index levels. Entry counts must start from the stated layout.

Check before moving on. Does an entry represent a record, a distinct value or a data block? Count index levels separately from the data fetch.

R29 B+ tree capacities

Draw one internal node with p child pointers and p-1 separators. If pointer bytes are P, key bytes K and available bytes B, require pP+(p-1)K <= B. Solve for the largest integer p. Leaf layout differs: account for each key/record pointer and any leaf-link overhead stated in the question.

With B=4096, P=10 and K=11, 21p-11 <=4096 gives p=195. A record pointer's size belongs in the leaf calculation, not this internal-node equation.

Worked example W29. An internal node with p child pointers has p−1 separator keys. With available bytes 1024, pointer size 4 and key size 8, 4p+8(p−1)≤1024 gives 12p≤1032 and p≤86. Check p=86: it uses 1024 bytes exactly. p=87 uses 1036 and fails. A leaf with 8-byte keys, 12-byte record pointers and a 4-byte next-leaf pointer satisfies 20q+4≤1024, so q=51. Child-pointer and record-pointer sizes play different roles. The earlier provisional answer 129 was incorrect and has been corrected.

Check before moving on. Test your proposed maximum by substitution, then show that adding one entry does not fit.

R30 Tree height and number of nodes

State whether height counts edges or levels. Compute leaf capacity and internal fanout separately. For the minimum possible height, fill nodes to maximum capacity and calculate how many leaves, parents and successive ancestors are needed. For occupancy bounds, apply minimum occupancy separately to nonroot nodes; the root has exceptions. Distinct indexed records are normally represented in B+ leaves; internal separators are not extra records. B-trees and B+ trees use different layouts.

Worked example W30. For 10,000 distinct records, maximum leaf capacity 50 and internal fanout 100, at least 200 leaves are needed. Their parents can be packed into 2 internal nodes, and one root points to those. Minimum height is 3 levels, or 2 edges. With only 5000 records, 100 leaves fit under one root, giving 2 levels. If a 3-level tree has minimum nonroot fanout 50 and minimum leaf occupancy 25, its root needs at least 2 children, giving at least 2×50×25=2500 records. A root that is itself a leaf uses its own exception. Do not apply the half-full rule to every root situation.

Check before moving on. State levels versus edges, leaf capacity, nonroot occupancy and root exceptions before multiplying.

R31 Tree insertion and deletion

Descend using separator comparisons. Insert in sorted leaf order. On overflow, split according to the stated convention and copy the appropriate leaf separator upward. Internal splits promote a separator rather than using the leaf copy rule. Propagate overflow up to the root and create a new root if necessary.

For deletion, remove the leaf entry, repair affected separators, then resolve underflow through the stipulated borrowing/merging rules. Propagate repair upward and shrink a redundant root. If the question leaves a consequential split or merge convention unspecified, identify the ambiguity instead of assuming all variants produce the same root.

Worked example W31. Use minimum-of-right-subtree separators, leaf capacity 2, internal fanout 3 and a leaf split with two entries left and one right. Start with root [7] and leaves [1,4] | [7,10]. Insert 2: split [1,2,4] into [1,2] | [4], copy 4 up, root becomes [4,7]. Insert 3: split [1,2,3] into [1,2] | [3], copy 3 up. The temporary internal keys [3,4,7] overflow. Promote middle key 4 and create root [4], internal children [3] | [7], leaves [1,2] | [3] | [4] | [7,10]. Delete 10: the right leaf [7] still meets minimum occupancy 1. Delete 7: it underflows, its sibling [4] cannot lend, so merge. The right internal node then has only one child; merge it with the left

internal node and collapse the old root. New root [3,4] points to leaves [1,2] | [3] | [4]. Other split conventions can produce other valid layouts; this example states its convention explicitly.

Check before moving on. Copy leaf separators, promote internal separators and repair root height. Follow the source's routing convention.

R32 Index choice and access cost

Hash indexes support equality lookup in the standard model. Ordered/B+ indexes also support ranges. Distinguish an equality join from a range predicate. For a composite ordered index, reason about the leading attributes; an arbitrary trailing-column predicate does not automatically get a direct range search.

Count index pages read on the path, subtract only explicitly cached levels, and add required data pages. A covering index can avoid fetching data pages. For ranges, include the relevant leaf pages and record pages; clustering changes the record-page cost. Heap scan, ordered-file binary search and hashing have different stated assumptions. Avoid one universal disk-access formula.

Worked example W32. A rating>3.4 predicate needs ordered access, so a B+ index supports it while a standard hash index does not provide a range search. GenreName='Comedy' is equality and can use either. Equality joins on identifiers can also use either under the simple exam model. A three-level B+ lookup with the root cached reads 2 remaining index pages and 1 data page: 3 I/Os. If the required columns are covered by the index, no data fetch is needed and the count becomes 2. A composite ordered index (Dept,Salary) directly supports a salary range within one fixed department; a predicate on Salary alone does not automatically provide the same leading-prefix search. Range costs also depend on matching leaf pages and whether matching records are physically clustered.

Check before moving on. Identify equality, range, leading-prefix use, caching and coverage. Count each page only once.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q55 · Equality versus range

Official question, PDF page 37. Published key: **A; B**.

The rating filter is a range and requires the B+ option in the standard model. Genre.Name='Comedy' is equality, so a hash or B+ index can serve it. The identifier links are equality joins. A uses B+ throughout; B uses a hash for Genre.Name while keeping ordered access for rating. C and D use hashing where the rating range is needed, so they fail. These are suitability claims, not guaranteed optimal query plans.

DA-2026-Q32 · Internal-node fanout

Official question, PDF page 21. Published key: **195 to 195**.

A node has p child pointers of 10 bytes and p−1 keys of 11 bytes. Require $10p+11(p-1) \le 4096$, or $21p \le 4107$. The maximum integer p is 195. At 196 pointers it uses 4105 bytes, exceeding 4096. The 12-byte record pointer is a leaf-layout detail, not an internal child-pointer size.

DA-2026-Q41 · Insertion with the printed separator convention

Official question, PDF page 26. Published key: **A**.

The printed root [5,8] sits above leaves [1,5],[7,8],[9,12], so its separators follow the left-child maxima, rather than W31's right-child minima. Insert 3 into the first leaf: it overflows to [1,3,5]. Splitting into [1,3] and [5] makes the temporary parent separators [3,5,8]. The internal node has too many children and splits around 5. The new root stores 5, giving A. Options with 3,5,8 describe a temporary overflowing node, not the repaired root.

CS-2024-set1-Q21 · Root occupancy

Official question, PDF page 15. Published key: **A**.

The ordinary half-full occupancy requirement has a root exception. A root can have only two children while the tree is small, or be a single leaf. Nonroot internal and leaf nodes still follow their stated minima. Only the root exception is correct: A.

CS-2025-set2-Q57 · Leaf split then parent split

Official question, PDF page 43. Published key: **33 to 33**.

The target leaf becomes [20,22,23,25,26]. With the smallest-right-key copy convention, split it into [20,22] and [23,25,26], copying 23 into parent [7,19,33,44]. The parent temporarily becomes [7,19,23,33,44]. Promote 23 in its internal split; the right parent has keys [33,44]. It is the parent of the leaf containing 25, and its smallest key is 33. The question asks for this parent key, not the root's smallest key.

CS-2023-Q62 · Sparse index and exact requested I/O

Official question, PDF page 65. Published key: **6 to 6**.

Unspanned 100-byte records give floor(1024/100)=10 per block, so 25,000 records use 2500 data blocks. The sparse index has 2500 anchors of 15+5=20 bytes. It fits 51 entries per index block and needs

ceil(2500/51)=50 blocks. Under the question's stipulated binary-search formula, ceil(log2 50)=6 accesses identify the target data block. Reading that data block would be an extra access, but is not requested. The downloaded file's PDF page is 65 even though its printed historical footer says 58.

Chapter practice

Original questions. Write the answer and the intermediate reasoning. Each lettered part belongs to the same problem. The separate solutions guide contains hints, steps and a final check.

P27 · NAT plus variation

Store 101 fixed 100-byte records unspanned in 1024-byte blocks. (a) Give records per block and required blocks. Why is ceil(10100/1024) wrong here? (b) Reserve a 50-byte header in every block and repeat.

P28 · Multilevel index

10,000 unspanned 100-byte records use 1000-byte blocks. Every index entry is 20 bytes at every level. (a) A sparse lowest level has one anchor per data block. Give block counts at each index level and uncached lookup I/Os including one data block. (b) Replace it with one dense entry per record and repeat.

P29 · Capacity

A B+ node has 1024 usable bytes; separator keys are 8 bytes and child pointers 4 bytes. Leaf entries have an 8-byte key plus a 12-byte record pointer, with a 4-byte leaf-link overhead. (a) Find maximum internal fanout and leaf capacity. (b) Double child-pointer bytes to 8; leave record pointers and leaf link unchanged. Recompute both capacities.

P30 · Height and occupancy

Maximum leaf capacity is 50 entries and maximum internal fanout is 100. (a) Find minimum height for 10,000 records and for 5000, both in levels and edges. (b) A 3-level tree has minimum nonroot fanout 50, minimum leaf occupancy 25 and minimum root fanout 2. Give its minimum and maximum record capacities.

P31 · Insertion and deletion

Use minimum-right-subtree separators. Leaves hold at most 2 keys and internal nodes at most 3 children. Split an overflowing 3-key leaf into 2 keys left and 1 right; split an overflowing 3-separator internal node by promoting its middle key. Start root [7], leaves [1,4] | [7,10]. (a) Insert 2, then 3; draw every tree state. (b) Delete 10, then 7. Nonroot leaves need at least 1 key and internal nodes at least 2 children. Borrow if a sibling can lend; otherwise merge and collapse a redundant root.

P32 · Choice and cost

(a) For each predicate, say whether standard hashing, an ordered B+ index, or both support a direct lookup: Rating>3.4; Genre='Comedy'; exact MovieID. (b) A B+ path has 3 levels and its root is cached. Count I/Os for one leaf lookup plus one data-page fetch, then for a covering lookup. A range touches 5 leaf pages and 6 data pages; count total I/Os without rereading the first leaf.

Before the exam

Draw the byte layout or tree state before using a formula. Distinguish separators from leaf records and data fetches from identifying a data block. State the height and routing conventions. Match the index to the predicate before discussing cost.